



Algorithms and Complexity 2026

Exercise Session 6

This is a collection of problems that hopefully improve your understanding of the material discussed in the lectures. During the exercise classes you have time to work on the first 3-5 problems and will have the opportunity to obtain feedback and/or help from one of the teaching assistants. Towards the end of each session the solutions of these problems will be covered.

To optimally prepare for the mastery test, you are encouraged to solve some of the remaining problems on your own. If you are stuck, require assistance or someone really messed up to write an understandable solution, feel free to send an email to algokomp-help@kth.se or visit an office hour.

Some problems are marked with superscripts: 1^M , 2^* or even 3^{**} . Problems marked with an **M** superscript are in the spirit of mastery test problems. Such problems are usually not stated as plain math problems but rather consist of a story from which you need to extract the mathematical structure yourself. Problems marked with a $\star$ are challenging problems. The more stars the harder – do not get discouraged if you cannot solve such a problem.

1 Las Vegas vs Monte Carlo

Suppose we have a Las Vegas algorithm $\mathcal{A}$ with expected running time $T(n)$ for a problem. In other words, $\mathcal{A}$ always outputs the correct answer, but may sometimes take a very long time to finish, but we know that for every input of size n to the problem, the expected number of time steps taken by $\mathcal{A}$ (over the random choices made by $\mathcal{A}$) is at most $T(n)$.

Show that there is a Monte Carlo algorithm for the same problem with guaranteed running time $O(T(n))$, and success probability at least 99% (in other words for every input, the algorithm finds the correct answer with probability at least 0.99 over the random choices made by the algorithm).

Solution:

The Monte Carlo algorithm operates as follows:

1. On an input x of size n , run $\mathcal{A}(x)$ for $100T(n)$ time steps, then stop it.
2. If $\mathcal{A}(x)$ terminated within $100T(n)$ time steps, output its answer.
3. Otherwise, output “don’t know”.

To see that this works, we need to prove that the probability that $\mathcal{A}(x)$ runs for more than $100T(n)$ steps is at most 0.01. Suppose for contradiction that this probability is some $p > 0.01$. But then the expected running time of $\mathcal{A}(x)$ would be at least $p \cdot 100T(n) > T(n)$, a contradiction. (Aside: instead of proving this fact from first principles we could have appealed to **Markov’s Inequality** which is a general fact along these lines.)

2 Approximate MAX k -CUT

The input to the optimization problem MAX k -CUT is a graph G . A solution is a partition of the vertices of G into k groups. The problem is to find a solution that maximizes the number of edges in the intersection between the groups, that is, between the vertices belonging to different groups. For $k = 2$ the problem is thus the same as MAX CUT discussed in Lecture 8.

Design a probabilistic approximation algorithm for MAX k -CUT and analyze the expected value of the objective function (i.e., the number of edges connecting vertices of different groups). The algorithm should produce a cut which in expectation cuts at least a $1 - 1/k$ fraction of all the edges in the graph.

Solution:

Let the algorithm for each of n vertices choose randomly which group it should belong to:

- 1: **for** $i \leftarrow 1, 2, \dots, n$ **do**
- 2: $group[i] \leftarrow random(1, k)$

For $e \in E$, let C_e be a random variable such that $C_e = 1$ if e is cut, and $C_e = 0$ otherwise. Notice that for $e \in E$, $\mathbb{P}(\text{"}e \text{ is not cut"}) = 1/k$. To see this, consider an edge $e = (u, v) \in E$. Then, " e is not cut" means that both u and v end up in the same group, and for any group assigned to u , v will be assigned to the same group with probability $1/k$.

Now, we define the random variable $C = \sum_{e \in E} C_e$, which counts the number of cut edges in the graph, and compute its expected value:

$$\begin{aligned}\mathbb{E}(C) &= \mathbb{E}\left(\sum_{e \in E} C_e\right) \\ &= \sum_{e \in E} \mathbb{E}(C_e) && \text{by linearity of the expectation.} \\ &= \sum_{e \in E} (1 \cdot \mathbb{P}(\text{"}e \text{ is cut"}) + 0 \cdot \mathbb{P}(\text{"}e \text{ is not cut"})) \\ &= \sum_{e \in E} \mathbb{P}(\text{"}e \text{ is cut"}) \\ &= \sum_{e \in E} (1 - \mathbb{P}(\text{"}e \text{ is not cut"})) \\ &= \sum_{e \in E} (1 - 1/k) \\ &= |E|(1 - 1/k)\end{aligned}$$

3 Faster Min-Cut

In Lecture 8 we covered the random contraction algorithm for the minimum cut problem, and arrived at a running time of $O(n^2 \cdot m)$ to solve the problem. Let us refer to this basic algorithm as ALGORITHM1. In this problem we will think of $m = \Theta(n^2)$ (i.e., a dense graph), for the sake of only having a single size parameter n to work with. Thus the running time of ALGORITHM1 is $O(n^4)$.

To improve this, it is useful to look at the analysis of the success probability: recall that the i 'th contraction step in the algorithm has probability approximately $1 - 2/(n - i)$ of succeeding. So the first steps, when i is small, has a very high chance of succeeding, but the final steps

have a quite high chance of failing. Guided by this intuition, we create the following algorithm, creatively named “ALGORITHM2”.

- 1: **function** ALGORITHM2(G)
- 2: $t \leftarrow$ parameter to be determined later, between 0 and $n - 2$
- 3: Run the random contraction algorithm on G , but only for $n - t$ contractions. I.e., we stop when there are t nodes left in the graph. Call the new graph with t nodes G' .
- 4: Return ALGORITHM1(G')

- (a) Analyze the success probability of ALGORITHM2 as a function of n and t . Given the success probability, how many times r should we run ALGORITHM2 in order to have a good chance to succeed?
- (b) Analyze the time complexity of a single run of ALGORITHM2, and the time complexity of r independent runs of the algorithm (for the value of r found in (a)).
- (c) How should we set t (as a function of n) in order to make the algorithm as fast as possible (while still having a good success probability)? What is the resulting overall time complexity as a function of n ?
- (d) Let us call the resulting algorithm (which runs ALGORITHM2 r times) ALGORITHM3. Now we could repeat this process: create a new algorithm which is identical to ALGORITHM2 except that it uses ALGORITHM3 rather than ALGORITHM1. If we repeat (a)-(c) for this new algorithm, what is the resulting overall time complexity?

Solution:

- (a) Let's write S the random event “the first $n - t$ contractions succeed”, and for $i \in \{0, \dots, n - t - 1\}$, S_i the random event “contraction i succeeds”. Since the first $n - t$ contractions succeed if and only if each S_i succeeds, we have $S = \bigcap_{i=0}^{n-t-1} S_i$. Then, using the same analysis as in the original analysis yields:

$$\begin{aligned}
 \mathbb{P}(S) &= \mathbb{P}\left(\bigcap_{i=0}^{n-t-1} S_i\right) \\
 &= \prod_{i=0}^{n-t-1} \mathbb{P}(S_i) && \text{as the events are independent} \\
 &= \prod_{i=0}^{n-t-1} \frac{n-2-i}{n-i} \\
 &= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{t-1}{t+1} \\
 &= \frac{t(t-1)}{n(n-1)} \\
 &\geq \frac{t^2}{2n^2}
 \end{aligned}$$

The last inequality holds as for $t \geq 2$, we have $t - 1 \geq t/2$, and hence $t(t - 1) \geq t^2/2$.

We set $p = \frac{t^2}{2n^2}$.

Now, we only need to fix the number of independent repetition r so that the probability of failure is small. Intuitively, the number of repetition r should depend on our probability of success p , so let's set $r = \alpha/p$ for some $\alpha \in \mathbb{N}$.

After r independent repetitions, the probability that every repetition failed is at most $(1-p)^r = ((1-p)^{1/p})^\alpha$.

Using the classic inequality $(1-p)^{1/p} \approx e^{-1}$ when p is small, we get that the probability of failing every repetition is less than $e^{-\alpha}$. Setting $\alpha = 50$ (and hence repeating the random contraction algorithm $r = 50 \cdot (2n^2/t^2)$ many times) yields that the probability of failure is less than $e^{-50} \approx 10^{-20}$ which is extremely small.

- (b) A single run: doing the contractions takes $O(n^2)$ time. Running Algorithm1 on G' takes $O(t^4)$ time, so we can bound the complexity by $O(n^2 + t^4)$.

For $r = \Theta(n^2/t^2)$ many runs, the time complexity thus becomes $O(n^4/t^2 + n^2 \cdot t^2)$.

- (c) Note that the first term in the time complexity decreases as t increases, and the second one increases. So to make the overall time complexity as small as possible, we should balance these terms and make them equal. Setting $n^4/t^2 = n^2 t^2$ we find that we should set $t = n^{0.5}$, which leads to an overall time complexity of $O(n^3)$.
- (d) Sub-problem (a) is unchanged. In sub-problem (b) the time complexity now becomes $O(n^4/t^2 + n^2 \cdot t)$, which in sub-problem (c) results in the best choice of t being $t = n^{2/3}$ and results in a running time of $O(n^{8/3}) \approx O(n^{2.67})$.

4 Generating Primes

In this task, we look at the problem of finding a prime number of a given size. To do this, you can take the following two facts for granted:

1. The total number of primes less than N is at least $\frac{7}{8} \cdot \frac{N}{\ln N}$ and at most $\frac{9}{8} \cdot \frac{N}{\ln N}$ (this is a concrete form of the **Prime Number Theorem**).
2. Given an n -bit integer N , there is an algorithm $\mathcal{B}$ that tests whether or not N is a prime in time $O(n^7)$.

Using these two facts, design and analyze a Las Vegas algorithm $\mathcal{A}$ which, given an integer n , outputs an n -bit prime N in expected running time $O(n^{10})$.

Solution:

The algorithm simply repeatedly generates a random n -bit number (i.e., a number N between 2^{n-1} and $2^n - 1$) and checks (using $\mathcal{B}$) whether or not it is prime, until a prime is found.

To analyze the algorithm we have to analyze the probability that the randomly chosen number is prime. Based on the fact given, the total number of n -bit primes is at least

$$\frac{7}{8} \frac{2^n}{\ln(2^n)} - \frac{9}{8} \frac{2^{n-1}}{\ln(2^{n-1})} = \frac{2^{n-1}}{8} \cdot \left(\frac{14}{n \ln(2)} - \frac{9}{(n-1) \ln(2)} \right) = \Omega(2^n/n)$$

There is a total of 2^{n-1} many n -bit integers, so we conclude that an $\Omega(1/n)$ fraction of these are primes.

In other words the probability that the randomly chosen integer in our algorithm is prime is $\Omega(1/n)$, which means that the expected number of random integers we have to generate until we find a prime is $O(n)$, leading to a total expected running time of $O(n^8)$.